

JTAG Enumeration for Security Research on the Zynq UltraScale+ MPSoC

1. Introduction

1.1 Why JTAG enumeration matters

JTAG is the most privileged hardware-level debug interface on a modern SoC. On an unhardened device, it provides:

- Read/write access to **every** memory location the SoC can address
- Halt and step control of **every** CPU (APU, RPU, PMU)
- Direct access to the security control unit's registers
- Read access to eFUSE shadow registers including device fingerprint, secure-boot policy, and PPK hashes
- The ability to release CPUs from reset and execute arbitrary code at **EL3** — the highest ARMv8 privilege level, above any OS, hypervisor, or TrustZone monitor

For an attacker with physical access, JTAG bypasses the operating system entirely. For a defender, JTAG state is what the silicon actually enforces — software-level hardening is irrelevant if the JTAG hardware gates are wide open.

The first question on any new device is: **what is the silicon actually gating, and what is it leaving open?** Answering it requires reading and decoding roughly eighty registers across seventeen functional sections — the security spine plus power, clocks, resets, A53 state, code-execution discovery, debug topology, memory map probes, peripheral and memory protection units, RPU configuration, IPC fabric, and FPGA configuration status — plus tracking the live state of every domain that could be a debug-relevant target. Doing this by hand is slow and error-prone. The enumeration tool described in Volumes 2 and 3 does it in ten seconds and produces a structured, diffable report.

2. Background

AMD's Zynq UltraScale+ MPSoC is the platform every step of this workflow targets. The subsections below cover its high-level architecture, variant landscape, boot model, JTAG security gates, and memory layout — in the order needed to understand what enumeration produces.

2.1 The ZynqMP SoC at a glance

The Zynq UltraScale+ MPSoC is a multi-die system-on-chip combining several processor subsystems with FPGA fabric:

Two power domains govern everything:

- **LPD** — Low-Power Domain. Always-on. Contains the RPU, PMU, CSU, OCM (on-chip SRAM), most peripherals

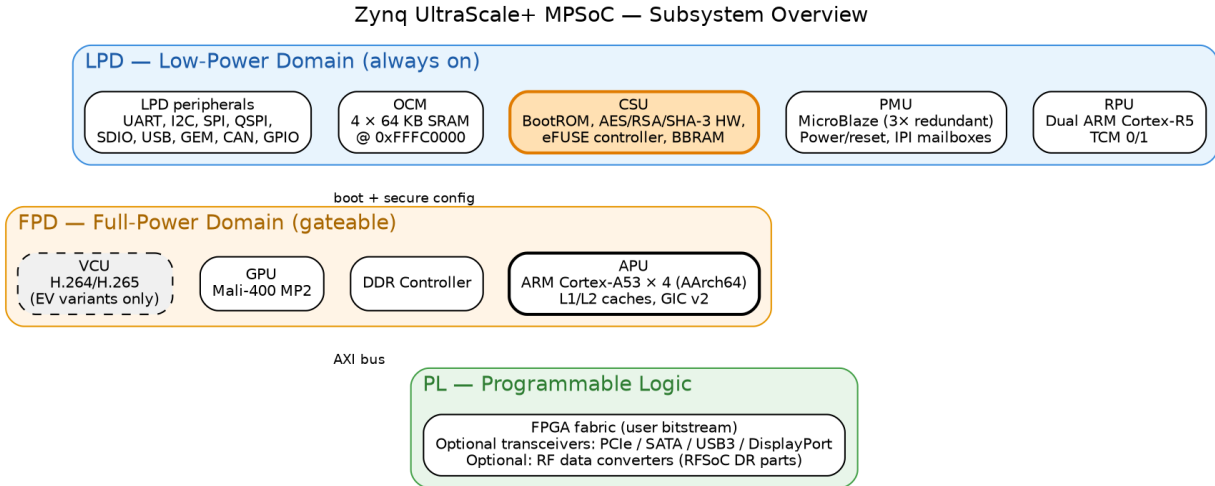


Figure 1: ZynqMP subsystem overview showing LPD, FPD, and PL domains

- **FPD** — Full-Power Domain. Can be gated off. Contains the APU, DDR controller, GPU, VCU, PCIe, SATA, USB3, DisplayPort, GIC

A device booted “JTAG-idle” with the FPD off can still service JTAG debug because the LPD remains powered.

2.2 Boot model

The ZynqMP boot sequence is multi-stage:

The JTAG-idle state is the cleanest baseline for enumeration: APUs are in reset, no FSBL has run, no secrets have been loaded into peripherals, no peripheral interrupts are active. Most of this paper’s sample data comes from a ZCU102 in JTAG idle.

A device booted from SD/QSPI/eMMC produces a much richer enumeration report because FSBL has run, DDR is initialized, PMU firmware is running, peripherals are alive, and U-Boot or Linux may be visible in memory. The two report types diff against each other to reveal what the FSBL chain actually did.

2.3 JTAG security architecture

This is the part that drives most of the rest of the series.

JTAG chain composition The ZynqMP exposes two TAPs in series on the JTAG chain:

The PS-TAP is Xilinx-proprietary; the ARM DAP is the standard CoreSight ARM debug-access port. Both must be reachable for full enumeration.

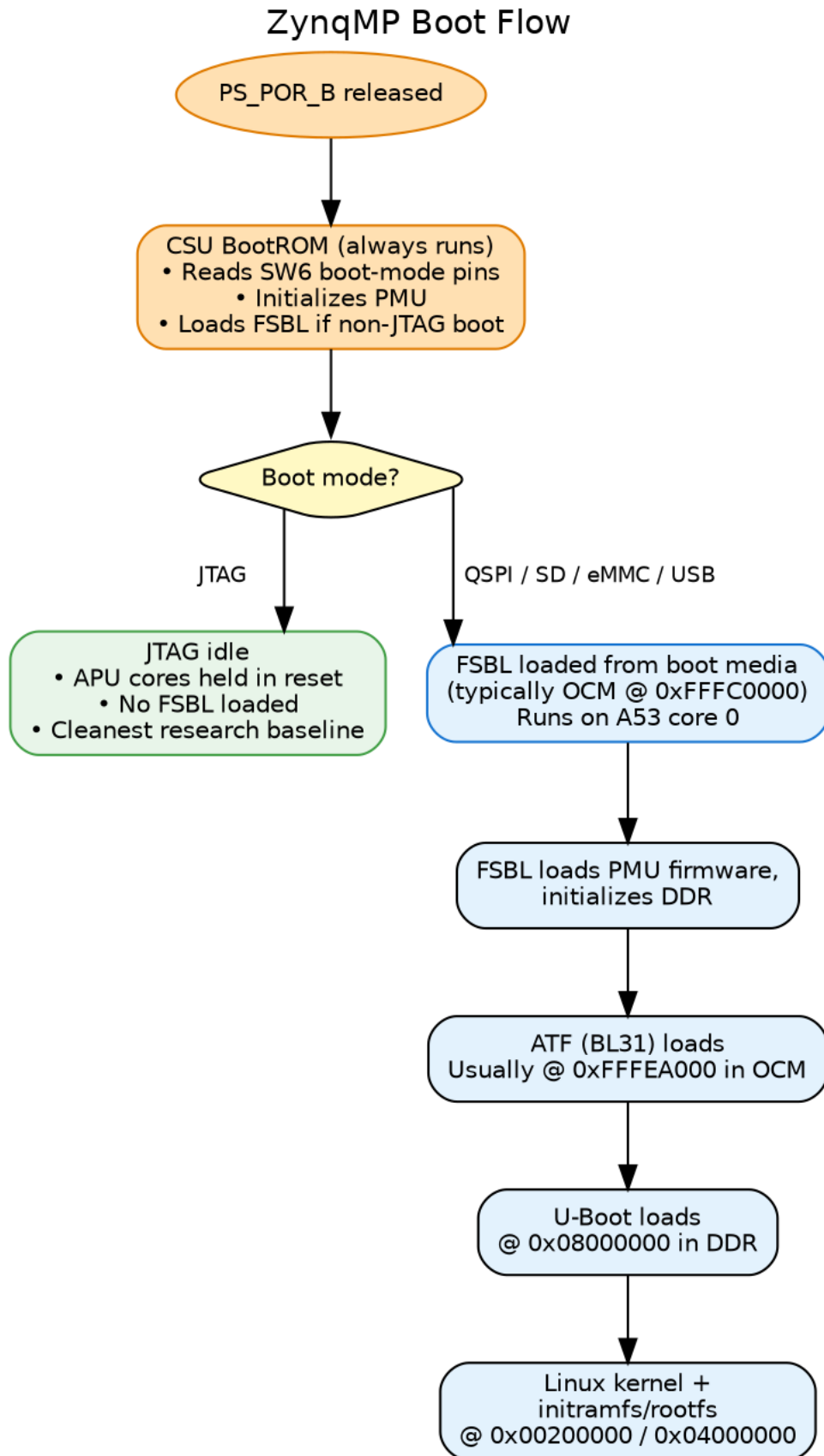


Figure 2: ZynqMP boot flow from PS_POR_B through Linux kernel load

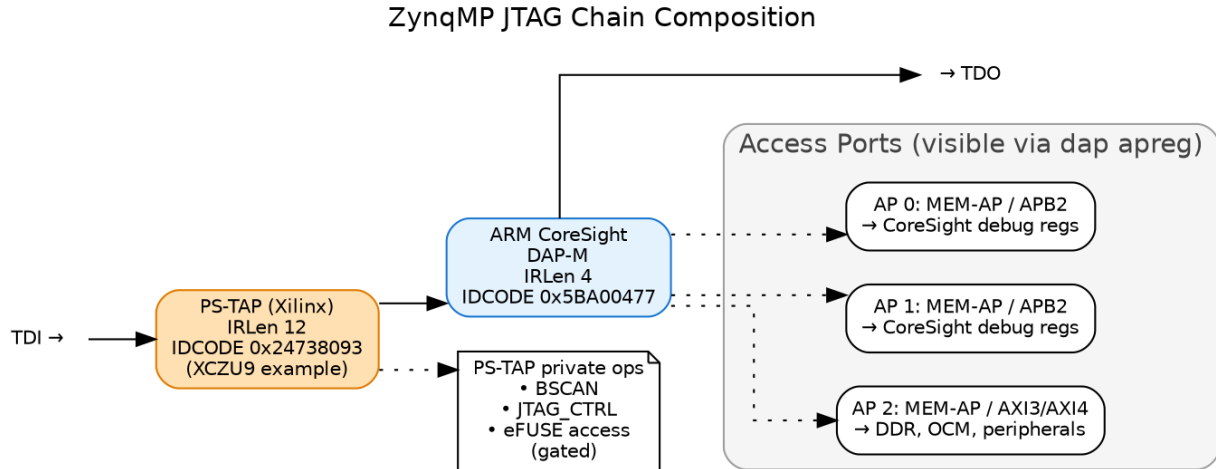
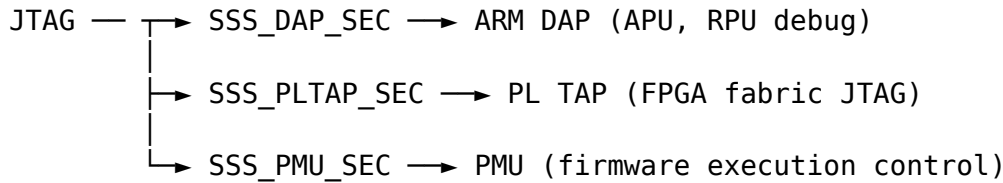


Figure 3: JTAG chain composition: PS-TAP + ARM CoreSight DAP with Access Ports

The CSU Secure Stream Switch (SSS) The CSU implements a multi-input multi-output stream switch that gates which JTAG paths can talk to which security-sensitive blocks. Three SSS paths matter for JTAG enumeration:



Each path's gate is controlled by a **3-bit magic value** in CSU.JTAG_SEC (at 0xFFCA0038). Open requires 0b111; any other value gates the path. The 3-bit design is anti-glitch — a single bit flip from fault injection cannot unlock a gated path.

Per-cluster debug authorization (JTAG_DAP_CFG) A second register CSU.JTAG_DAP_CFG (at 0xFFCA003C) controls what the ARM DAP itself can do, with six 1-bit gates:

Bit	Field	Effect
0	SSSS_APU_DBGEN	APU halt/step/breakpoint allowed (non-secure world)
1	SSSS_APU_NIDEN	APU non-invasive trace (ETM, etc.) allowed
2	SSSS_APU_SPIDEN	APU SECURE-world debug allowed (EL3/TrustZone)
3	SSSS_APU_SPNIDEN	APU secure-world trace allowed
4	SSSS_RPU_DBGEN	RPU (R5) debug allowed
5	SSSS_RPU_NIDEN	RPU trace allowed

These are **cluster-wide**, not per-core. SPIDEN is the bit that defines whether the device’s secure-world is attackable via JTAG.

eFUSE secure-boot policy Finally, the one-way fuses in EFUSE.SEC_CTRL (at 0xF-FCC1058) control the silicon’s secure-boot policy at hardware level:

Bit(s)	Field	Effect when set
0	AES_RDLK	AES key in eFUSE cannot be read
1	AES_WRLK	AES key in eFUSE cannot be written
2	ENC_ONLY	Boot image MUST be encrypted
3	BBRAM_DIS	BBRAM disabled as key source
5	JTAG_DIS	Hardware JTAG disable — irreversible
10	SEC_LOCK	Lock SEC_CTRL itself — no further fuses settable
[25:11]	RSA_EN	15-bit magic field requiring specific value to enforce RSA-signed boot
26	PPK0_WRLK	Primary Public Key 0 hash write-locked
[28:27]	PPK0_INVLD	2-bit magic field; invalidates PPK0
29	PPK1_WRLK	PPK1 hash write-locked
[31:30]	PPK1_INVLD	2-bit magic field; invalidates PPK1

Two fields use magic-value patterns (RSA_EN, PPK_INVLD) rather than single-bit enables — the same anti-glitch defense as the JTAG_SEC gates. A successful glitch attack on a single bit cannot enable RSA bypass or invalidate a PPK.

These three register groups — JTAG_SEC, JTAG_DAP_CFG, and EFUSE.SEC_CTRL — define essentially the entire JTAG-visible security policy of the device. Reading them is the heart of the enumeration workflow described in Volume 2.

2.4 Memory layout overview

ZynqMP’s address space (identical across MPSoC variants):

Range	Block	Notes
0x00000000 - 0x7FFFFFFF	DDR (lower 2 GB)	Main system DRAM. Only accessible after DDR controller init by FSBL.
0xC0000000 - 0xFCFFFFFF	PL aperture / PCIe	Programmable Logic and PCIe BARs
0xFD000000 - 0xFD0FFFFF	DDR controller	FPD Clock/Reset (APLL/DPLL/VPLL, RST_FPD_*) Gigabit transceivers RVBARADDR0-3 reset vectors. APU power/reset.
0xFD1A0000 - 0xFD1AFFFF	CRF_APB	
0xFD400000 - 0xFD4FFFFF	SERDES	
0xFD5C0000 - 0xFD5CFFFF	APU registers	
0xFD800000 - 0xFD9FFFFF	GPU (Mali-400)	EV variants only
0xFE000000 - 0xFE0FFFFF	GIC distributor	
0xFE100000 - 0xFE1FFFFF	VCU (video codec)	
0xFF000000 - 0xFF0FFFFF	PS UARTs, I2C, SPI, GPIO, QSPI	
0xFF160000 - 0xFF17FFFF	SDIO0/1	PMU↔APU↔RPU communication
0xFF300000 - 0xFF33FFFF	IPI mailboxes	
0xFF410000 - 0xFF41FFFF	LPD_SLCR	
0xFF5E0000 - 0xFF5EFFFF	CRL_APB	
0xFF9A0000 - 0xFF9AFFFF	RPU configuration	CSU at 0xFFCA0000 DNA, SEC_CTRL, PPK hashes Battery-Backed RAM (AES key storage)
0xFFC80000 - 0xFFCBFFFF	CSUDMA + CSU	
0xFFCC0000 - 0xFFCCFFFF	eFUSE controller	
0xFFCD0000 - 0xFFCDFFFF	BBRAM	
0xFFCE0000 - 0xFFCEFFFF	RSA core	Hardware RSA-2048/4096 Power/reset of every domain PMU firmware runs here Tightly-Coupled Memory for R5 cores Default FSBL/ATF load region
0xFFD80000 - 0xFFD8FFFF	PMU_GLOBAL	
0xFFDC0000 - 0xFFDDFFFF	PMU RAM (LMB)	
0xFFE00000 - 0xFFE3FFFF	RPU TCM	
0xFFFC0000 - 0xFFFFFFFF	OCM (4×64 KB SRAM)	

This layout is identical across ZCU102/104/106, Ultra96, custom ZynqMP boards, and RFSocS — the variant-specific differences (VCU, RF tiles, GPU presence) are bonded selections within these address ranges, not different memory maps.

JTAG Enumeration for Security Research on the Zynq UltraScale+ MPSoC

Volume 2 — Methodology and Workflow

This volume describes how the enumeration toolset is structured (the short methodology preface) and what running it actually looks like in practice (the longer workflow section). A reader who only wants to know “what do I do with this thing” can skim §1 and dive into §2.

A ZCU102 case study with real report excerpts is in §3.

1. Methodology Preface

The workflow is shaped by three design decisions. Understanding them makes the rest of the document make sense and points at where future extensions slot in.

1.1 Three-layer pipeline: discover → enumerate → deep-probe

Each layer is more invasive than the previous. Discovery is pure metadata reads; enumeration reads ~80 SoC registers passively; deep-probe touches large memory regions and may risk DAP wedges. The layering keeps fast safe operations from being held hostage by slow risky ones.

The **enumerate** layer itself is split into two stages: a Tcl capture stage (`openocd/enumerate.tcl`, the only code that talks to silicon) and a Python interpret stage (`tools/interpret.py`, which applies curated annotations + cross-register rules to the captured JSON). The two stages are described in §2.3 (Capture) and §2.4 (Interpret).

1.2 QEMU as the authoritative source of register layouts

Bit-field decodings in the enumeration script come from Xilinx’s QEMU register models, not from hand-typed datasheet transcriptions.

The C-language REG32/FIELD macros in QEMU’s source are “as-implemented” — they describe how the silicon actually responds, not how a datasheet text describes it. By making them the canonical source, the script cannot drift from real-silicon behavior through human transcription error.

Originally a standalone audit tool (`tools/audit-bit-layouts.py`) cross-checked every hand-typed `dump_reg` call against the QEMU model. After all sites migrated to `dump_reg_qemu` (which pulls fields directly from the QEMU dict at call time), the audit had nothing to audit and was retired in May 2026 — drift is now structurally impossible.

Discover → Enumerate → Deep-Probe Layering

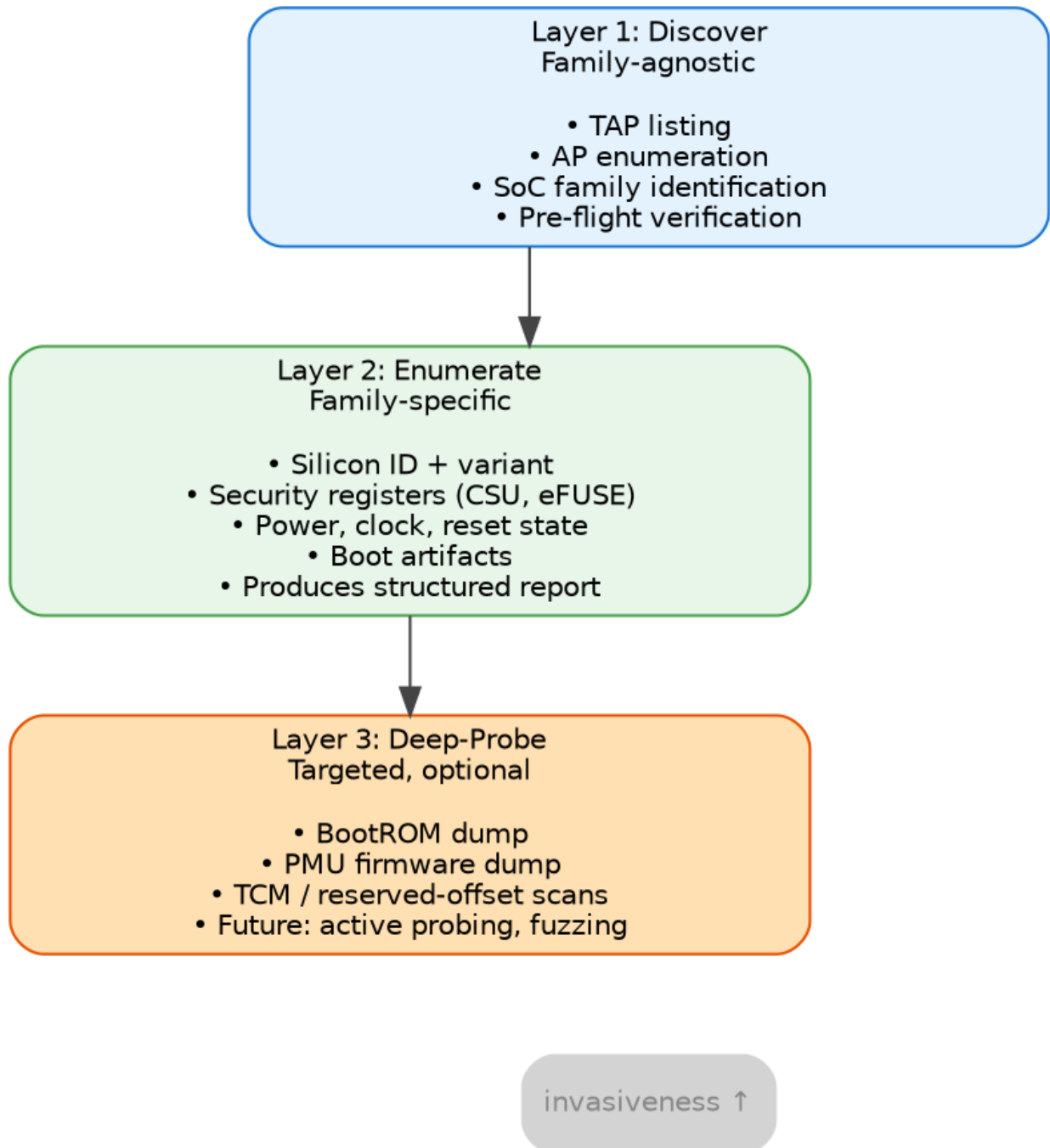


Figure 4: The three-layer enumeration pipeline

QEMU as Authoritative Source of Bit Layouts

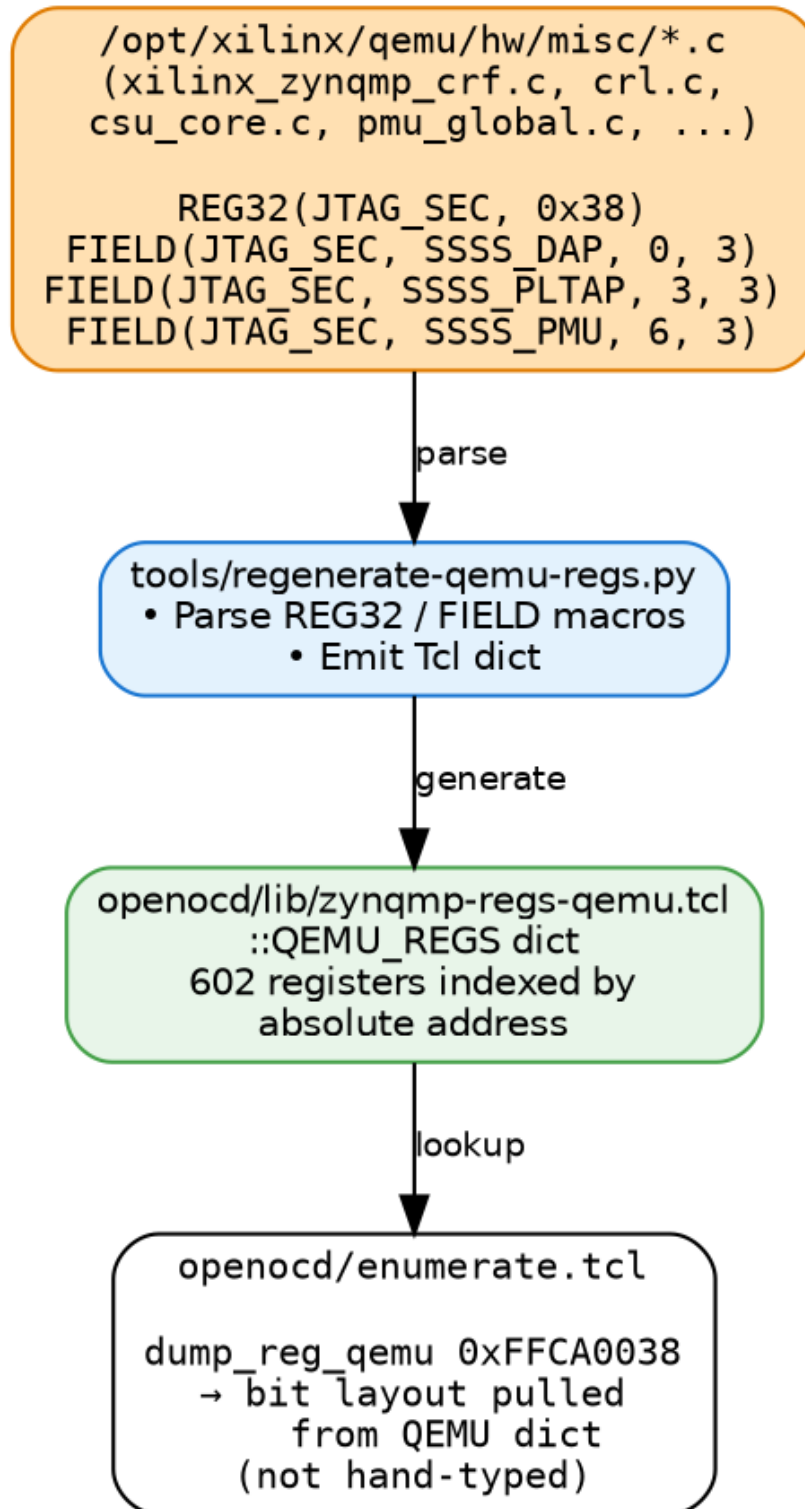


Figure 5: QEMU sources → generator → Tcl dict → enumerate.tcl lookup

1.3 Variant lookup for within-family differences

The ZynqMP family includes dies with 2 or 4 A53 cores, some with VCU, some with RF tiles, some with reduced GEM counts. The enumeration script reads the IDCODE PART_ID and consults `openocd/lib/zynqmp-variants.tcl` (25 entries) to obtain a capability profile for the specific die. Variant-conditional probes use this profile to avoid reading addresses that aren't present on the detected die.

This pattern is preferable to per-variant scripts (which would duplicate the same ~1100 lines of code N times) and to per-board scripts (which would conflate “this die has VCU silicon” with “this PCB exposes a VCU connector”). One script, one register library, one variant table.

2. Workflow

This is the hands-on section. The workflow has five stages for a first-contact unknown-board scenario; on a known board it collapses to three steps (the auto-detect stage is skipped).

2.1 Stage 1 — Connect

Concrete steps for a ZCU102:

Step	Action	Verify
1	Set boot mode switches SW6 to all-ON (JTAG idle) — for the cleanest baseline	Visual check
2	Connect 12V power supply to the board	Green PWR LED on
3	Connect the on-board Digilent SMT2 JTAG over a USB Micro-B cable to the host	Host sees new USB device
4	If running in a VM, pass the JTAG USB device through to the guest	<code>lsusb</code> in guest shows 0403:6014
5	Press the POWER ON button (SW1)	Status LEDs come up

For other boards the principle is the same: bring up power, connect JTAG, pass USB through.

2.2 Stage 2 — Confirm the chain

Work targets a known board, so the OpenOCD config is fixed (`openocd/zcu102.cfg` — the Digilent SMT2 interface plus the ZynqMP target). Before trusting an enumeration,

First-Contact Enumeration Workflow

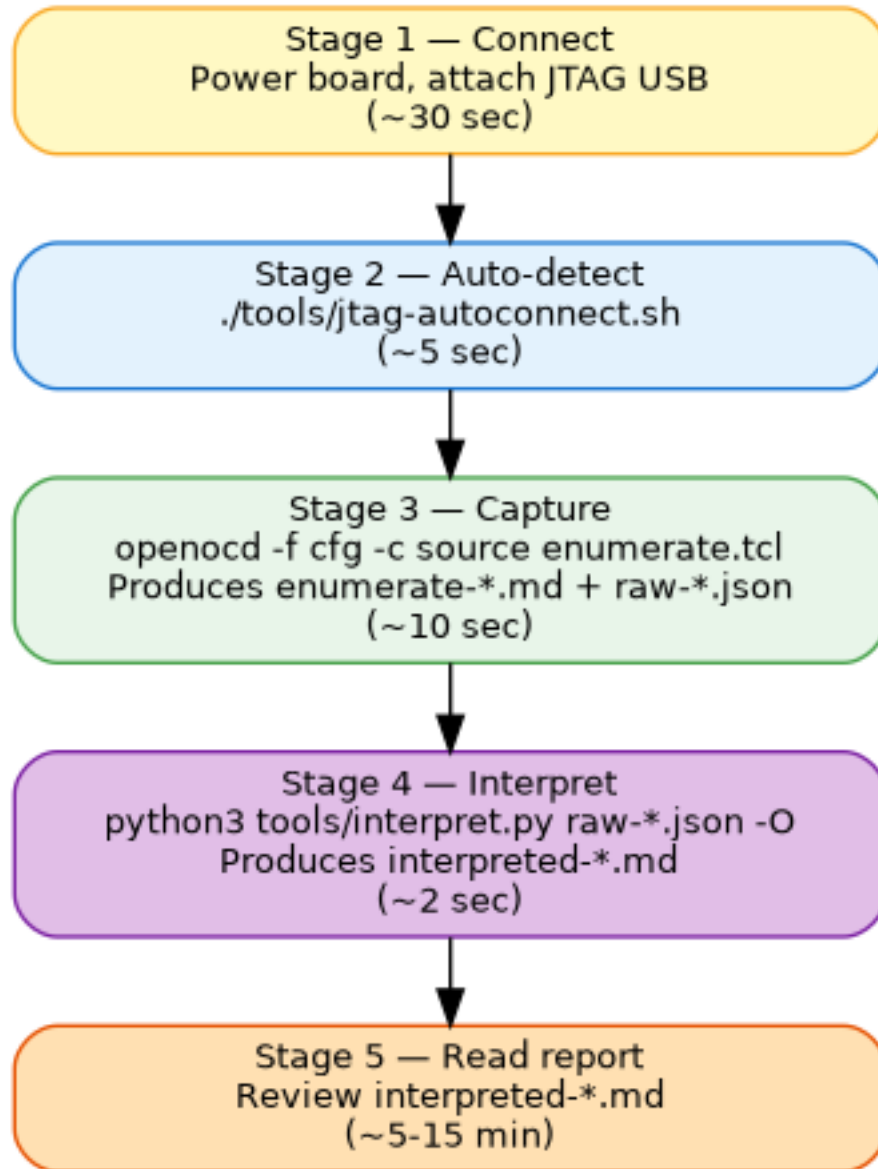


Figure 6: Five-stage workflow: Connect → Auto-detect → Capture → Interpret → Read report

optionally run `discover.tcl` to confirm the JTAG chain and AP topology match what you expect for the part in front of you:

```
openocd -f openocd/zcu102.cfg -c "init; source openocd/discover.tcl; shutdown"
```

```
-----
JTAG CHAIN DISCOVERY
-----
TAPs defined: 2
- uscale.tap
- uscale.ps
Access Ports visible on uscale.dap:
AP 0: IDR = 0x44770002 MEM-AP APB2/APB3 (debug-register access)
AP 1: IDR = 0x44770002 MEM-AP APB2/APB3 (debug-register access)
AP 2: IDR = 0x24770004 MEM-AP AXI3/AXI4 (memory access)
-----
```

The TAP list and IDCODEs confirm the part; the AP list shows which debug/memory access ports respond. On a different target board a mismatch here is the first signal that the config (or the part) isn't what you assumed — which AP responds is itself a posture datum.

2.3 Stage 3 — Capture (run `enumerate.tcl`)

The capture is one command:

```
openocd -f openocd/zcu102.cfg -c "init; source openocd/enumerate.tcl; shutdown"
```

In about ten seconds it produces three outputs:

Output	Where	Use
Raw markdown	reports/enumerate- <YYYY-MM-DD-HHMMSS>.md	Human-readable transcript — addresses + per-bit decode for every register read
Raw JSON	reports/raw-<YYYY-MM-DD-HHMMSS>.json	Structured capture for the interpret stage (§2.4). Schema: registers, variant, a53, boot_state, memory_probes, coresight, metadata
Live terminal	stdout	Real-time progress; includes OpenOCD startup banner and per-section streaming output

The raw markdown and the raw JSON cover the same data in two formats. The interpreted markdown (produced in §2.4) is layered on top of the JSON.

A clean run has these phases (visible in terminal):

OpenOCD startup:

- JTAG chain scan, IDCODEs reported
- Target examination (A53 cores held in reset → expected warnings)

Section-by-section enumeration:

```
$1 JTAG Chain (no probes – metadata)
$2 Silicon Identity (IDCODE, version, DNA, variant lookup)
$3 Boot State (boot mode pins, reset reason, multi-boot)
$4 Security State (JTAG_SEC, JTAG_DAP_CFG, EFUSE.SEC_CTRL –
the security spine)
$5 Power State (PWR_STATE, error history)
$6 Clocks (PLLs, per-peripheral REF_CTRLs)
$7 Reset State (per-domain and per-peripheral resets)
$8 A53 Release (writes to RST_FPD_APU to release core 0, halts in EL3)
$9 Code Execution (OCM scan, conditional DDR scan)
$10 CoreSight Topology (per-AP `dap info N` capture into JSON; each AP's ROM-
table walk + component PIDs)
$11 Memory Map Reference (documentation table embedded in the report)
$12 Memory Map Probe (block accessibility check)
$13 XPPU (LPD peripheral protection – CTRL, ISR, master-
ID slots)
$14 RPU Configuration (Cortex-R5 cluster – global mode, per-
core CFG/STATUS/SLV_BASE)
$15 IPI (APU agent window) (inter-processor interrupt fabric from APU's view)
$16 XMPU (OCM_XMPU + DDR_XMPU0 – memory-range protection)
$17 PL TAP + PCAP (FPGA configuration status – PL_DONE, PL_EOS, etc.)
Cleanup (re-asserts A53 reset for next-run cleanliness)
```

2.4 Stage 4 — Interpret (run interpret.py)

The capture stage produced a raw JSON; the interpret stage layers findings + annotations on top of it. One command:

```
python3 tools/interpret.py "$(ls -t reports/raw-*.json | head -1)" -0
```

Produces reports/interpreted-<YYYY-MM-DD-HHMMSS>.md. The -0 flag auto-names the output to match the raw JSON's timestamp.

The interpreted report contains:

Section	What's in it
Silicon identity table	Die, variant, DNA, silicon revision — from variant lookup
Findings (rules fired)	Each rule that fired gets a coloured severity glyph (CRITICAL / MAJOR / MINOR / INFO — markdown viewers render these as red/orange/yellow/blue circles), description, conclusion prose, and offensive implications

Reading a finding Each fired rule in the interpreted report looks like this:

```
### [CRITICAL] Secure-world TrustZone debug enabled
```

```
_SPIDEN bit is set – JTAG can halt and inspect the EL3 monitor /  
TrustZone secure world._
```

```
**Conclusion:** Secure-world debug is the single most consequential  
JTAG security signal on ZynqMP. With SPIDEN=1, an attacker with  
physical JTAG access can halt the A53 in EL3 (highest ARMv8  
privilege)...
```

```
**Offensive implications:**
```

- Direct read of AES decryption keys from secure-world memory
- Direct read of attestation/identity material protected by TrustZone OS
- Halt and modify the EL3 monitor – TrustZone rootkit primitive
- ...

(In an actual markdown viewer, the [CRITICAL] placeholder is rendered by `interpret.py` as a red filled circle; the PDF representation here strips the glyph for font portability.)

The severity glyph, the rule name, the one-line description, the conclusion prose, and the offensive-implications list all come from the corresponding `rule_*` function in `docs/findings/zynqmp_rules.py`.

3. Case Study: ZCU102 in JTAG-Idle Mode

This section walks through real enumeration output from a ZCU102 dev board in JTAG-idle (cleanest baseline). Excerpts are taken directly from a saved report file; everything is reproducible from a fresh power-on.

3.1 Silicon identity

Raw register dump (§2):

- CSU_IDCODE: 0xffca0040 = 0x24738093
 - [0] CONST_1 = 1
 - [11: 1] MANUF_ID = 0x49 (Xilinx, JEP106)
 - [27:12] PART_ID = 0x4738 (XCZU9 die)
 - [31:28] REVISION = 0x2 (silicon rev 2)
- CSU_VERSION: 0xffca0044 = 0x00000513
 - [15:12] PLATFORM = 0x0
 - [3: 0] PS_VERSION = 0x3 (production silicon)

eFUSE Device DNA (96 bits):

- DNA[31:0]: 0x44804345
- DNA[63:32]: 0x0170cfa7
- DNA[95:64]: 0x40000000

Findings table:

Observation	Observed	Implication
Chip identity	XCZU9 (EG/CG) (4-core A53, family=zynqmp)	PART_ID 0x4738, silicon rev 2. Quad A53; ZCU102 ships with ZU9EG — no VCU on this die
Silicon variant	PS_VERSION=3 → production silicon	Production is common; ES1/ES2 = early revision with possible errata
Device DNA	0x44804345 0x0170cfa7 0x40000000	Unique per-chip ID; useful for fingerprinting

Confirms: ZU9EG die, silicon rev 2, production silicon (not an engineering sample), and the DNA we'll fingerprint against. The “no VCU on this die” note comes from the variant lookup table — even if the package suffix were EV, this specific die doesn't bond VCU silicon.

3.2 Boot state

- CRL_APB.BOOT_MODE_USER: 0xff5e0200 = 0x00000000
 - [15:12] ALT_BOOT_MODE = 0x0
 - [8] USE_ALT = 0
 - [3: 0] BOOT_MODE = 0x0 (JTAG idle)
- CRL_APB.RESET_REASON: 0xff5e0220 = 0x00000001
 - [0] EXTERNAL_POR = 1
 - ... (all other bits 0)

Findings table:

Observation	Observed	Implication
Boot mode	0 → JTAG idle	Cleanest research baseline. No FSBL ran. APU cores in reset. Only BootROM executed.
Reset reason	EXTERNAL_POR	Fresh baseline — POR only, no prior software state.

We're in the cleanest possible state for enumeration.

3.3 Security state — the most important section

- CSU.JTAG_SEC: 0xffca0038 = 0x0000003f
 - [8: 6] SSSS_PMU_SEC = 0x0 (PMU path gated)
 - [5: 3] SSSS_PLTAP_SEC = 0x7 (PL TAP path open)
 - [2: 0] SSSS_DAP_SEC = 0x7 (DAP path open)
- CSU.JTAG_DAP_CFG: 0xffca003c = 0x000000ff
 - [5] SSSS_RPU_NIDEN = 1
 - [4] SSSS_RPU_DBGGEN = 1
 - [3] SSSS_APU_SPNIDEN = 1 ← secure-world TRACE enabled
 - [2] SSSS_APU_SPIDEN = 1 ← secure-world DEBUG enabled
 - [1] SSSS_APU_NIDEN = 1
 - [0] SSSS_APU_DBGGEN = 1

Findings table (abbreviated):

Observation	Observed	Implication
APU invasive debug (DBGGEN)	1	Cluster-wide gate. 1 = halt/step/breakpoint allowed.
APU non-invasive trace (NIDEN)	1	1 = ETM/trace output allowed without halting.
APU SECURE invasive debug (SPIDEN)	1	WIDE OPEN — secure-world TrustZone debug is enabled. Can halt/inspect EL3 monitor, secure-world OS. Expected 0 on hardened production.
APU SECURE non-invasive trace (SPNIDEN)	1	Secure-world trace enabled — can observe TrustZone execution without halting.
RPU invasive debug (DBGGEN)	1	1 = R5 cluster debug allowed
RPU non-invasive trace (NIDEN)	1	1 = R5 trace allowed
DAP path (CSU SSS → ARM DAP)	0x7	UNLOCKED — secure stream switch to DAP open.
PLTAP path (CSU SSS → PL JTAG)	0x7	UNLOCKED — secure stream switch to PL TAP open.
PMU path (CSU SSS → PMU)	0x0	Gated/partially gated.

eFUSE secure-boot policy:

- EFUSE.SEC_CTRL: 0xffcc1058 = 0x00000000
All bits clear: AES_RDLK=0, AES_WRLK=0, ENC_ONLY=0, BBRAM_DIS=0,
JTAG_DIS=0, SEC_LOCK=0, RSA_EN[25:11]=0, etc.

Observation	Observed	Implication
EFUSE.SEC_CTRL overall	0x00000000	Factory/dev default for an evaluation kit. No security fuses blown.
JTAG_DIS	0	JTAG enabled at hardware level (we got this far, so this had to be 0)
RSA_EN	0	No RSA signature verification of boot images
ENC_ONLY	0	Unencrypted boot images allowed
SEC_LOCK	0	SEC_CTRL still mutable; one-way bits could still be blown
AES_RDLK / AES_WRLK	0 / 0	AES key in BBRAM/eFUSE can be read and written
Hardened device baseline	expect RSA_EN magic field set, SEC_LOCK=1, possibly JTAG_DIS=1	This is what you'd compare against for a production device

Summary of security state. Secure-world TrustZone debug is fully open. APU and RPU debug are unrestricted. DAP and PLTAP paths to the CSU stream switch are fully unlocked. The PMU path is gated, which means the script can't directly debug the PMU from JTAG — but the PMU's runtime state is still readable via memory reads. eFUSE is in the factory-default state: nothing has been blown. This is the maximum-access dev-kit posture.

3.4 A53 release succeeded

A53.0 state: halted

PC = 0x00000000fffc0000 (our safe-landing instruction in OCM)

CPSR = 0x000003cd (EL3H, AArch64, all interrupts masked)

Findings:

Observation	Observed	Implication
A53 core 0 state	halted at PC = 0xFFFFC0000, CPSR mode = EL3H	EL3H = Exception Level 3, Handler mode — the highest ARMv8 privilege. Above any OS, hypervisor, or TrustZone monitor. Can read/write any memory and any system register.
Research capabilities unlocked	halt, single-step, memory-write at EL3	Inject arbitrary code, dump TrustZone-protected memory once enabled, install rogue EL3 handlers, observe secure-world execution.

This confirms our minimum-viable execution control on the device: A53 core 0 is halted in EL3 with no FSBL, no PMU firmware, no ATF. From this state we can:

- Read any address the AXI mem-AP can reach
- Write to any writable region
- Set arbitrary RVBARADDR values
- Single-step through code we load
- Eventually capture system registers via stage-2 payloads (deferred)

3.5 What the report tells us in aggregate

For this ZCU102 dev kit in JTAG-idle:

- **Hardening posture:** factory dev kit. Every security gate is open. No eFUSES have been blown.
- **Attack surface available right now:** full SoC memory, all CSU registers, eFUSE shadow, BBRAM, PMU memory (read-only via memory reads since PMU path is gated for direct debug), APU and RPU debug control, secure-world (TrustZone) debug.
- **What the workflow proved:** the toolset can identify the device, decode the security registers correctly, and release a core into a controlled EL3-halt state — all without any vendor toolchain.

The next step would be to compare this against a production-hardened device to see which of these capabilities survive. That comparison is the empirical heart of the research direction this series enables.

JTAG Enumeration for Security Research on the Zynq UltraScale+ MPSoC

Volume 3 — What Enumeration Reveals (and How to Use It)

Enumeration over JTAG is a **security-configuration assessment**: read the registers where ZynqMP’s security controls live, and their values tell you which controls are on. This volume is the field guide. Each indicator category lists the exact attributes and **where to find them in a register dump** (block · address · bits), how the dev-board baseline compares to a hardened part, and what the observed state enables.

The reference target is the ZCU102 (XCZU9EG, S/N 210308BD8D4D): factory silicon with **no security provisioned** — the **all-open baseline**, where every control reads OFF/unprovisioned. The payoff is comparison: point the same enumeration at another (hardened) board and the deviations from this baseline are the story. The exhaustive flat catalog of every attribute is [../11-enumerated-attributes.md](#); this volume is the interpretive companion to it.

1. How to read this volume

Every category has the same two parts:

- **Attributes & locations** — a table of *what* each attribute is, *where* it lives (BLOCK.REGISTER · 0xADDRESS · bits), its value on this dev board, and what a hardened part shows instead. The address+bits column is what lets you locate the attribute in a raw register dump.
- **What it enables** — the concrete techniques the observed state allows, with prerequisites and limits.

Read every value against the **hardening continuum** — a reading that’s unremarkable on a dev kit is significant on a production device, and vice-versa:

Factory	→	Dev kit	→	Partial hardening	→	Production hardened
unblown		blown but		blown + partial		all gates closed,
fuses		mutable		locking		keys baked, JTAG off

Scope. This assumes the debug threat model — JTAG/DAP is reachable. On hardened silicon that access is itself closed (JTAG_DIS or debug authentication), so “JTAG can read X” only applies where JTAG is open by design or by misconfiguration. No fault injection, side-channel, or supply-chain pre-positioning.

The fastest read on a new board is the **Security Posture Summary** (§10) — a one-row-per-control checklist that flips OFF/dev → ON/provisioned.

2. Silicon identity

Attributes & locations

Attribute	Where (block · address · bits)	This board	Hardened part
Device part / family	CSU.IDCODE · 0xFFCA0040	0x24738093 (XCZU9EG)	identifies die + revision
Silicon revision	CSU.VERSION · 0xFFCA0044 (PS_VERSION)	3 (production)	0/1 = ES1/ES2 sample
Per-chip unique ID	EFUSE.DNA_0..2 · 0xFFCC100C–1014	0x40000000_0170CFA7_44804345	read-gated

What it enables

- *Pre-engagement recon.* Exact die + revision tells you which errata apply, which boot-image format the BootROM expects, and what prior exploit work exists.
- *Engineering-sample detection.* An ES revision in a shipped product is worth escalating — ES silicon often carries security errata fixed in production steppings.
- *Fleet fingerprinting.* DNA is unique per chip and readable from the eFuse shadow; across a fleet it builds a DNA→identity table. DNA readable on a part that should lock it is itself a finding.
- *Variant inference.* If the variant table claims a subsystem (e.g. VCU) but its base address times out — or vice-versa — the package bond differs from the catalog.

3. Secure-debug exposure (the most consequential indicator)

Attributes & locations

Attribute	Where (block · address · bits)	This board	Hardened part
APU secure invasive debug	CSU.JTAG_DAP_CFG · 0xFFCA003C · bit 2 (SPIDEN)	1 (open)	0
APU secure trace	0xFFCA003C · bit 3 (SPNIDEN)	1 (open)	0
APU non-secure debug	0xFFCA003C · bit 0 (DBGEN)	1	0 or 1
APU non-secure trace	0xFFCA003C · bit 1 (NIDEN)	1	0
RPU debug / trace	0xFFCA003C · bits 4,5 (RPU DBGEN/NIDEN)	1	0
DAP JTAG path	CSU.JTAG_SEC · 0xFFCA0038 · bits [2:0] (DAP_SEC)	0b111 (open)	not 0b111
PL-TAP JTAG path	0xFFCA0038 · bits [5:3] (PLTAP_SEC)	0b111 (open)	not 0b111

Attribute	Where (block · address · bits)	This board	Hardened part
PMU JTAG path	0xFFCA0038 · bits [8:6] (PMU_SEC)	0b000 (gated)	gated

The whole JTAG_DAP_CFG reads 0xFF and JTAG_SEC reads 0x3F on this board — maximum debug exposure. Each JTAG_SEC field needs the magic 0b111 to unlock (anti-glitch: a single-bit flip can't open it).

What it enables

- *TrustZone inspection (SPIDEN=1)*. Halt the A53 in EL3 and read everything the secure world holds — keys in OCM/DDR, the EL3 monitor, attestation material.
- *Secure-world key extraction*. Keys the BootROM/ATF stage into secure RAM are directly readable; eFuse-shadow keys too, if not separately gated.
- *EL3 implant*. Write a secure region, redirect the secure vector table into it, resume — the EL3 monitor then runs attacker code on every SMC.
- *Secure trace (SPNIDEN=1, SPIDEN=0)*. Watch secure execution live without halt control — map the SMC surface, see when/where keys load.
- *Non-secure debug (DBGEN=1, SPIDEN=0)*. Halt and patch the kernel/RTOS, disable checks, escalate privilege.
- *Pre-attack confirmation*. SPIDEN=1 makes expensive attack development (fault injection, supply-chain) unnecessary — confirm the easy path first.

4. eFuse secure-boot policy

Attributes & locations

Attribute	Where (block · address · bits)	This board	Hardened part
RSA boot authentication	EFUSE.SEC_CTRL · bits [25:11] (RSA_EN)	0 (off)	15-bit magic = enforced
Encrypt-only boot	0xFFCC1058 · bit 2 (ENC_ONLY)	0	1
JTAG disable	0xFFCC1058 · bit 5 (JTAG_DIS)	0	1
Secure lockdown	0xFFCC1058 · bit 10 (SEC_LOCK)	0	1
PPK0 public-key hash	EFUSE.PPK0_0..11 · 0xFFCC10A0–10CC	all-zero	non-zero (root of trust)
PPK1 public-key hash	EFUSE.PPK1_0..11 · 0xFFCC10D0–10FC	all-zero	non-zero

Attribute	Where (block · address · bits)	This board	Hardened part
PPK0 / PPK1 revoked	0xFFCC1058 · [28:27] / [31:30] (PPK_INVLD)	0	2-bit magic = revoked

Field widths are anti-glitch defenses (RSA_EN is 15 bits, each PPK_INVLD 2 bits) — a single-bit fault can't enable a bypass or revoke a key, so a *partial* magic pattern is a red flag.

What it enables

- *Boot-chain enforcement profile.* SEC_CTRL = 0 ⇒ unsigned/unencrypted boot accepted (full boot-chain control). RSA magic only ⇒ signature required. RSA + ENC_ONLY ⇒ signed-encrypted only.
- *Image substitution.* On a low/partial-enforcement target, swap in attacker firmware. A partial profile (RSA_EN set, SEC_LOCK clear) is still mutable.
- *AES key-extraction feasibility.* AES_RDLC = 0 means the eFuse AES key is readable — recover it and all “encrypted” images decrypt offline (see §5).
- *PPK-revocation bypass.* A partial PPK_INVLD pattern can mean an incomplete key rotation — the BootROM may still trust a key meant to be revoked.
- *Fleet outlier hunting.* Inventory SEC_CTRL across a product line; the less-hardened units are the higher-value targets.

5. Key material and confidentiality

Attributes & locations

Attribute	Where (block · address · bits)	This board	Hardened part
AES key-slot population	CSU.AES_STATUS · 0xFFCA1000 · bits 8–11 (per-slot * _ZERO)	0xF00 (all slots empty)	a slot's bit clear = key loaded
eFuse AES key present	EFUSE.EFUSE_AES_CRC · 0xFFCC0048	0 (absent)	non-zero (key burned)
AES key read-lock	EFUSE.SEC_CTRL · 0xFFCC1058 · bit 0 (AES_RDLC)	0 (readable)	1
AES key write-lock	0xFFCC1058 · bit 1 (AES_WRLK)	0	1
PUF provisioned	EFUSE.PUF_CHASH · 0xFFCC1050	0 (unprovisioned)	non-zero

What it enables

- *Key-model determination.* The combination says whether the device decrypts boot with an eFuse AES key, a BBRAM key, or a PUF-derived key — which decides the extraction approach.
- *Direct key disclosure.* “Key present (AES_CRC $\neq$ 0) + AES_RDLK = 0” is the flag: the boot-decryption key is unprotected from a debug read. Recover it → decrypt every encrypted image for that device, or forge new ones.
- *PUF parts.* A PUF-provisioned part derives the key at boot rather than storing it; extraction shifts to helper-data handling, but the enumeration still tells you which class of attack applies.

6. Anti-tamper

Attributes & locations

Attribute	Where (block · address · bits)	This board	Hardened part
Tamper response policy	CSU.CSU_TAMPER_0..12 · 0xFFCA5004–5034	All-zero (disarmed)	non-zero per armed source
Tamper events latched	CSU.TAMPER_STATUS · 0xFFCA5000	0 (none)	non-zero = a source fired
Software tamper trigger	CSU.CSU_TAMPER_TRIG0 · 0xFFCA0014		—

Each CSU_TAMPER_n register wires one tamper source to a response (system reset / secure lockdown / BBRAM + key zeroize).

What it enables

- *Tamper-aware operation.* The config registers show which physical / voltage / temperature / JTAG-activity events are armed — i.e. what *not* to trip on a live target, since a response may zeroize keys or lock the device down.
- *Response-gap hunting.* A device that arms some sources but leaves the consequential ones (key zeroize) unconfigured is a softer target.
- *Forensics.* Latched TAMPER_STATUS bits on a fielded unit are evidence a source already fired.

7. Memory and peripheral TrustZone

Attributes & locations

Attribute	Where (block · address · bits)	This board	Hardened part
DDR TrustZone (ch. 0–5)	DDR_XMPU0..5.CTRL/LOCK · 0x00000000...0x00000000	0x00000000 · permit	default-region + LOCK set
FPD TrustZone	FPD_XMPU.CTRL/LOCK · 0x00000000	—	configured
OCM TrustZone	OCM_XMPU.CTRL · 0x00000000	0x3	regions configured
Peripheral protection	XPPU.CTRL/ISR · 0x00000000	enabled, no violations	apertures locked down

ZynqMP enforces memory/peripheral TrustZone with XMPU (ranges) and XPPU (apertures) — there is no single TZASC.

What it enables

- *Cross-master reachability.* XMPU/XPPU config tells you whether a foothold on one master (PCIe, USB, a released R5) can reach secure DDR/OCM. Default/open XMPU means a compromised master can read secure memory.
- *Misconfiguration as a path.* A permissive region — or one with LOCK clear, so reconfigurable — is a direct route into protected memory.
- *PL-TAP exposure.* With PLTAP_SEC = 0b111 (§3) the FPGA JTAG TAP is reachable — read/modify configuration memory, extract or load bitstreams.

8. Boot and runtime state

Attributes & locations

Attribute	Where (block · address · bits)	This board	Hardened part
Boot mode	BOOT_MODE_USER · 0xFF5E0200 · bits [3:0]	0 (JTAG idle)	SD/QSPI/eMMC/USB
Multiboot search offset	CSU.CSU_MULTI_BOOT · 0xFFCA0010	0	image-chaining offset
Auth / encrypt status	CSU.CSU_STATUS · 0xFFCA0000 (BOOT_AUTH/BOOT_ENC)	0	set after authenticated boot
Boot artifacts (when booted)	OCM 0xFFFFC0000 (FSBL), DDR 0x08000000 (U-Boot), 0x00200000 (kernel), ...	n/a in JTAG-idle	present + dumpable

What it enables

- *Strategy by state.* JTAG-idle is a clean slate (only BootROM ran). A booted device has FSBL/U-Boot/kernel up and secrets in DDR — larger but state-dependent.
 - *Live firmware extraction.* Once an artifact is located, JTAG reads it straight from physical memory, bypassing software read-protection (`dump_image uboot.bin 0x08000000 0x100000`). Version strings in the dump drive version-specific targeting.
 - *ATF/BL31 patch.* If ATF is at `0xFFFFEA000` (OCM) or in DDR, the EL3 monitor is readable and — with SPIDEN + write access — patchable.
 - *MULTI_BOOT image switching.* Writing `CSU_MULTI_BOOT` + reset changes which image the BootROM loads, given boot-media write access.
-

9. Power, clock, reset

Attributes & locations

Attribute	Where (block · address · bits)	This board	Hardened part
Powered domains	<code>PMU_GLOBAL.PWR_STATE</code> · <code>0xFFD80100</code>	Only BootROM-era domains up	per running stack
Peripheral clocks	per-peripheral <code>*_REF_CTRL</code> (e.g. <code>0xFF5E0050+</code>)	most gated in idle	per running stack
Reset state	<code>RST_FPD_APU</code> · <code>0xFD1A0104</code> , <code>RST_LPD_I0U2</code> · <code>0xFF5E0238</code> , ...	cores/peripherals in reset	cleared per running stack
A53 reset-vector	<code>APU.RVBARADDR0L</code> · <code>0xFD5C0040</code>	—	(used by the release recipe)

What it enables

- *The A53 release primitive.* Release A53 core 0 over raw JTAG with no FSBL: set the landing pad in `RVBARADDR0L`, clear the `RST_FPD_APU` reset bits (ACPU0 / L2 / PWRON), halt — the core lands in EL3. From there: load code, read any memory, modify the EL3 monitor.
 - *Peripheral bring-up.* Clear a peripheral's reset bit + configure its `*_REF_CTRL` to bring it online (e.g. UART for console capture, or to drive a device).
 - *Live attack-surface map.* A peripheral both clocked and out-of-reset (UART, GEM, USB) is directly targetable — GEM with a live stack means network reach.
 - *Selective denial.* Asserting resets or powering down domains (PL, GEMs, OCM, APU) is a denial primitive — useful when the defender's response runs on the same SoC.
-

10. The Security Posture Summary

`tools/interpret.py` distills §2-§7 into a single **Security Posture Summary** (`rule_security_posture_summary`): one row per control, each with its location, current value, and an OFF/dev → ON/provisioned verdict. Rows shown in **bold** are the ones that most change exposure when ON — RSA enforcement, encrypt-only, JTAG_DIS, AES read-lock, PPK provisioned, secure debug, tamper armed.

Read it first on any new board: all rows OFF means an all-open part; any deviation is the lead to pursue. The exhaustive per-attribute detail behind each row is in [../11-enumerated-attributes.md](#).

11. Future work

- **Hardened-device comparison (the unblocking step).** The payoff is diffing this all-open baseline against a known-hardened ZynqMP — which controls read differently and which capabilities survive each hardening fuse. Acquiring such a part is the highest-value next move.
 - **Live per-core debug-authentication read.** The posture currently *synthesizes* debug authorization from JTAG_DIS/DFT_DIS + SPIDEN/SPNIDEN + DBGEN. A booted-state read of each core's CoreSight DBGAUTHSTATUS would confirm the live signal (needs core examination, so it stays out of the JTAG-idle baseline to avoid DAP-wedge risk).
 - **AP-topology capture.** Fold `discover.tcl`'s AP enumeration into the report — which APs respond is itself a posture datum (a hardened part may gate the AXI mem-AP while leaving the APB debug AP open).
 - **R5 BootROM dump.** Wake the R5 in booted state and dump from a different bus-master ID than the A53 (the A53-EL3 path is chip-filtered).
-

12. References

AMD / Xilinx documentation

- **UG1085** — Zynq UltraScale+ Device TRM (memory map, security model, boot flow, CSU/PMU/eFuse).
- **UG1087** — Register Reference (per-register bit decode).
- **UG1182** — ZCU102 Evaluation Board User Guide (switches, connectors, USB IDs).
- **UG1137** — MPSoC Software Developer Guide (boot chain, FSBL, ATF).

Open-source code (authoritative bit layouts)

- github.com/Xilinx/qemu — the REG32/FIELD register models that are the source of truth for `openocd/lib/zynqmp-regs-qemu.tcl` (`hw/misc/csu_core.c`, `xilinx_zynqmp_crf.c/crl.c/pmu_global.c`, `hw/nvram/xlnx-zynqmp-efuse.c`).
- github.com/Xilinx/u-boot-xlnx — `arch/arm/mach-zynqmp/include/mach/hardware.h` register structs.

- github.com/Xilinx/embeddedsw — FSBL / xilsky / xilsecure address + key-handling definitions.

ARM

- CoreSight Architecture Specification (DAP, APs, DBGGEN/NIDEN/SPIDEN/SPNIDEN semantics).
- ARMv8-A Architecture Reference Manual (exception levels, secure-world system registers).

Tooling

- OpenOCD 0.12+ (openocd.org); JEP106 (JEDEC manufacturer IDs for IDCODE decode).

Internal references

- [docs/11-enumerated-attributes.md](#) — the per-attribute catalog behind this volume.
- [docs/05-enumeration-tool.md](#) — the `enumerate.tcl` (18 sections) + `interpret.py` pipeline.
- [openocd/lib/zynqmp-regs-qemu.tcl](#) — verified register/address/bit source of truth.
- [docs/annotations/zynqmp_security.py](#), [docs/findings/zynqmp_rules.py](#) — decoders + posture rules.

End of Volume 3.